

# AGV Fleet Monitor 3D & Digital Twin



A Cyber-Physical System Integrating Edge Microcontrollers, Real-Time WebSockets, and Three.js 3D Virtualization.

**PROJECT:** Physical AI - ABA Fusion (Semaine 2)

**FINAL DELIVERY:** Jour 4 (Technical Report)

**DEVELOPED BY:** Hafida Belayd & Abdelkhalek Hanbel

---

## 1. Executive Summary

Modern warehouse automation relies extensively on Automated Guided Vehicles (AGVs) to coordinate logistics. Managing a physical fleet of AGVs presents challenges, particularly around system visibility, network latency, and collision prevention. This project resolves these issues by implementing a cyber-physical digital twin architecture.

By combining an embedded ESP32 microcontroller, a Python simulation routing broker, and an interactive React Three Fiber 3D client, we provide operators with real-time telemetry streaming, visual route coordinates, and manual trajectory overrides. Security is enforced via a two-layer safety mechanism combining automated hardware interrupts and cloud coordinated software locks.

## 2. System Architecture

The system uses a four-layered Cyber-Physical Architecture designed to ensure decoupling, reliability, and real-time visualization. Data flows from sensors up to the browser visualizer and down from the user HUD as commands.

## SYSTEM ARCHITECTURE (DIAGRAMME D'ARCHITECTURE)

### Embedded Edge Layer (ESP32 Microcontroller)

Reads HC-SR04 ultrasonic sensor, controls L298N motor driver, updates local I2C LCD displaying custom status icons.

↕ (SSL MQTT Port 8883 - JSON/Text Payloads)

### Communication Bridge (HiveMQ Cloud Broker)

Brokers telemetry topics ("twin2/telemetry") and command queues ("robot/control") over secure TLS.

↕ (Background Telemetry Queue Worker Threads)

### Simulation & Routing Engine (Python Backend Server)

Computes Dijkstra path planning, manages Zone X Mutex lock, hosts WebSocket server.

↓ (10Hz WebSocket JSON Broadcast)

### 3D Presentation HUD (Vite + React + R3F)

Renders animated AGVs with rotating LIDARs and dynamic pointLights, displays glassmorphic status controls.

## 3. Embedded Edge Layer (ESP32 Firmware)

---

The physical vehicle is governed by an ESP32 edge microcontroller executing a non-blocking scheduling loop. The firmware maintains strict timing for telemetry updates, LCD refreshes, and distance scans to maximize responsiveness.

### 3.1. Non-Blocking Design and LCD Layout

Unlike legacy looping structures that rely on `delay()`, our firmware uses `millis()` timer comparison thresholds. This ensures the MQTT callback and loop processing run at full speed. An I2C LCD screen (1602) displays live operational metrics.

To elevate visual feedback, we registered custom character binary matrices directly to the LCD memory during ``setup()`` to render icons:

-  (Robot icon) displayed next to active speed values.
-  (Battery symbol) displayed next to battery levels.
-  (Heartbeat symbol) that blinks dynamically every 500ms when connected to the broker.
-  (Hazard symbol) that flashes rapidly at 250ms intervals if an obstacle is detected in the safety range.

### 3.2. Startup Animation

During the initialization phase inside ``setup()``, the LCD runs a visual loading sequence. It prints a progress bar block-by-block across the second line (16 segments total) and displays `"Booting AGV OS... ` before flashing `"System Active! `.

## 4. Software & Coordination Layer (Python Backend)

---

The Python server (running `headless_simulator.py`) manages fleet physics and navigation logic. It handles path routing and coordinates traffic intersections to prevent collisions.

## 4.1. Dijkstra Path Planning

The warehouse is defined as a graph containing node coordinates (Zones A, B, C, D, Charging Zone R) and lanes. When a new target is dispatched, the backend computes the shortest path using Dijkstra's algorithm and streams coordinates step-by-step.

## 4.2. Traffic Coordination (Zone X Mutex)

Zone X is a central intersection prone to collisions. Access is governed by a software **\*\*Mutex Lock\*\*** ( `ZoneXTrafficController` ). If AGV-01 occupies the zone, a lock is acquired, and any secondary AGVs requesting access are placed in a `WAIT` state. Once AGV-01 exits, the lock is released, and the next vehicle proceeds.

# 5. Real-Time 3D Digital Twin Visualizer

---

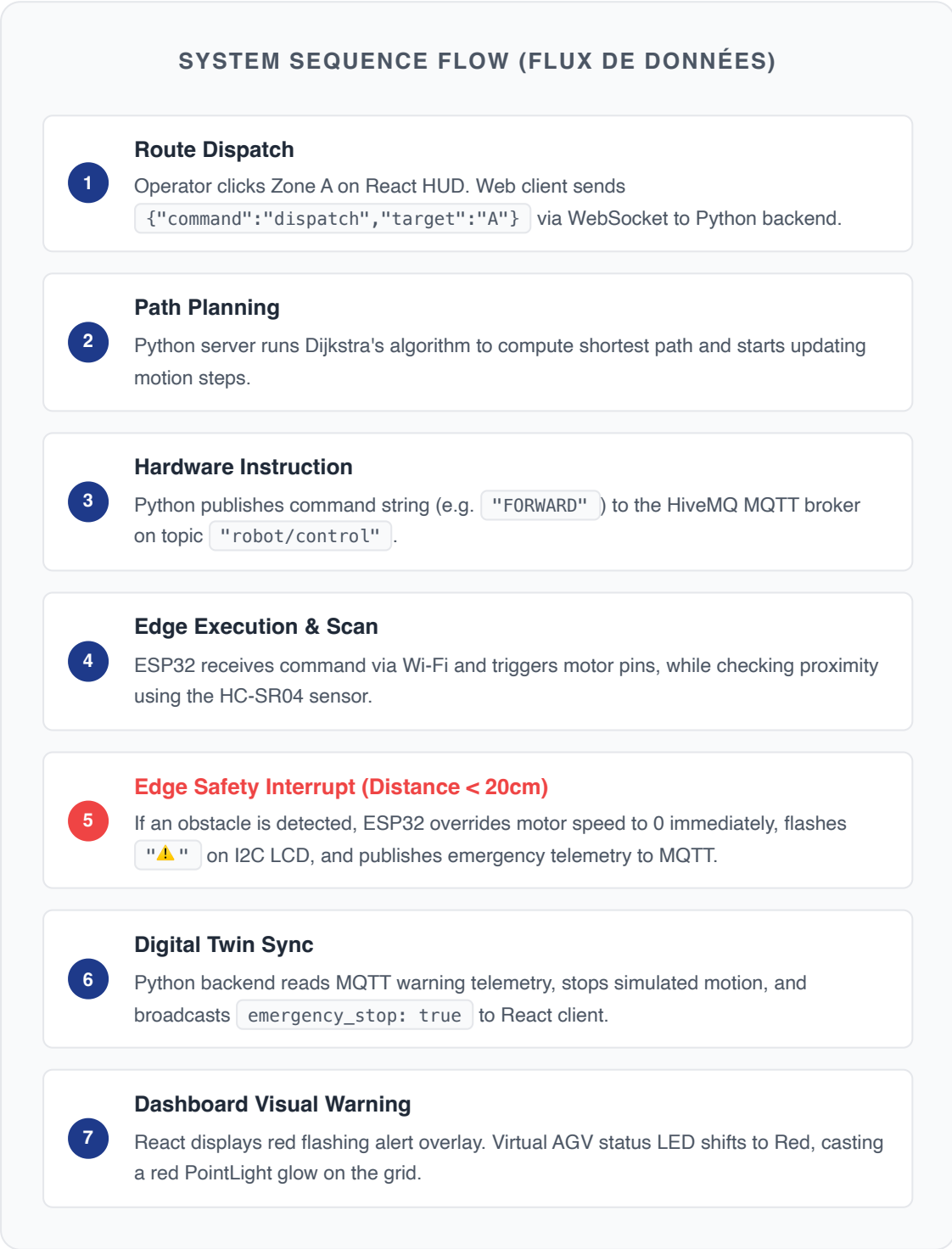
The React frontend connects via WebSockets to the Python backend to synchronize the virtual and physical states. It features an interactive **\*\*Glassmorphism HUD\*\*** panel built with CSS backdrop-filters, custom Outfit/Space Grotesk typography, and neon glows.

Inside the Three.js 3D canvas (React Three Fiber), the visual assets have been enhanced to match the physical states:

- **Spinning LIDAR:** A 3D LIDAR cylinder is rendered on top of the AGV model and spins continuously using the animation loop.
- **Dynamic Status LEDs:** An LED emitter changes colors based on the AGV's state: Green (moving), Orange (waiting/yielding), Red (emergency stop), and Rose (critical intersection).
- **Floor PointLight Projection:** A PointLight projects status-colored light on the floor grid under the AGV, creating real-time visual shadows.
- **Emissive Nodes:** Blue glowing spheres indicate junction coordinates.

## 6. Cyber-Physical Sequence & Data Flow

The diagram below traces the communication lifecycle when an operator triggers a route dispatch, highlighting the edge-level safety interrupts.



## 7. Security and Safety Failsafes

---

### CRITICAL SAFETY SYSTEMS

To prevent runaway movement, the ESP32 implements a **1500ms Watchdog Timer**. If the Python backend fails to send commands within this window, the ESP32 automatically halts all motors. Furthermore, if Wi-Fi or MQTT connections are lost, the robot cuts motor power instantly before executing reconnect logic.

## 8. Conclusion

---

By integrating edge hardware safety overrides with an interactive 3D virtual twin, this project delivers a highly secure and traceable AGV fleet coordination platform. The low-latency MQTT/WebSocket synchronization and real-time visual warnings ensure safe industrial automation.